CS6400-A Project Milestone (Group 18)

Latency-Aware Adaptive Reranking for Hybrid RAG Retrieval

Wei-Po Lin
wlin401@gatech.edu
Georgia Institute of Technology

Yu-An Chen
ychen3606@gatech.edu
Georgia Institute of Technology

You-Hsiang Tsai
ytsai309@gatech.edu
Georgia Institute of Technology

Siang-Yu Cheng
scheng371@gatech.edu
Georgia Institute of Technology

1 PROGRESS CHECK IN

1.1 Implementation Progress

We break the project into eight components. Five are complete and three have not started.

Component	Status	Notes
Dataset processing	Complete	Parsing, chunking, metadata, provenance IDs
Evidence mapping	Complete	Gold evidence text mapped to paragraph and chunk IDs
Sparse retrieval (BM25)	Complete	Original implementation, standard library only
Dense retrieval	Complete	`BAAI/bge-m3` embeddings, FAISS exact index
Retrieval evaluation	Complete	Recall@K, MRR, hit rate, latency; reproducibility manifests
RRF hybrid fusion	Not started	Next component
Adaptive reranking	Not started	Cross-encoder plus confidence gating
Answer generation	Not started	QA accuracy is a secondary metric

Dataset processing. We prepare QASPER v0.3 into four artifacts: source paragraphs, retrieval chunks, questions, and reference annotations. Chunks are 180 words with 30 words of overlap and never cross a paragraph boundary, so every chunk carries the identifier of the paragraph it came from. The train split yields 888 papers, 47,770 paragraphs, 50,189 chunks, and 2,593 questions. Every build records the input checksum, the chunking parameters, the counts, and the output checksums in a manifest, and the scripts refuse to overwrite existing outputs. This works as expected: an independent rebuild of the train split on a second machine reproduced the recorded corpus checksum exactly.

Evidence mapping. QASPER annotates gold evidence as paragraph text rather than identifiers, so a separate step matches each evidence string back to a prepared paragraph. Matches are classified as exact, whitespace-normalized, ambiguous, figure or table, section heading, unmatched, or empty. Only unambiguous matches on answerable annotations become retrieval positives. The step also emits an audit file of every entry that needs attention, which is what our eligibility policy in Section 2.1 is built from. Working as expected.

Sparse retrieval. BM25 with $k_1 = 1.5$ and $b = 0.75$ over a Unicode word tokenizer, implemented from scratch rather than through a library, with deterministic tie-breaking by chunk identifier. Index construction takes 1.9 seconds over 50,189 chunks and 47,344 distinct terms; retrieving the top 100 chunks for all 2,593 questions takes 26.4 seconds. Working as expected.

Dense retrieval. `BAAI/bge-m3` embeddings in a FAISS `IndexFlatIP` index over normalized vectors, so scores are exact cosine similarities with no approximation. The builder refuses to run if any chunk would exceed the encoder's token limit; the longest QASPER chunk is 1,061 tokens against a limit of 8,192, so nothing is truncated. Working as expected.

Retrieval evaluation. A single evaluator scores any number of run files in one invocation, which forces every compared system to share one eligible question set, one chunk-to-paragraph mapping, and one latency estimator. It verifies the full provenance chain by checksum — corpus to index to run — and refuses to score runs produced from a different query set. Three regression tests cover the cases that would silently corrupt the results table. Working as expected.

1.2 Division of Work

Member	Role	Completed work
1. Yu-An Chen	Data and Preprocessing	Repository setup, QASPER parsing and chunking, evidence mapping, preprocessing documentation, exploratory data analysis
2. Wei-Po Lin	Retrieval and Indexing	BM25 index construction and retrieval, sparse pipeline tests
3. Siang-Yu Cheng	Reranking and Adaptive Policy	Dense embedding generation, FAISS index construction, dense retrieval
4. You-Hsiang Tsai	Evaluation and Generation	Evaluation harness, retrieval metrics, latency measurement, baseline experiment runs, result analysis, reproducibility manifests

The roles are those assigned in the proposal, with one scheduling adjustment. Both required baselines were built in parallel: Member 2 implemented sparse retrieval while Member 3 implemented dense retrieval, rather than Member 2 building both in sequence. This brought the milestone's hard requirement forward and let evaluation begin earlier. Member 3 now moves to the reranking and adaptive policy that the proposal assigns to that role, and Members 2 and 3 jointly own RRF fusion, which sits between their two components.

1.3 Progress Assessment and Remaining Work

Measured against what our proposal committed to deliver by the milestone — a working preprocessing pipeline, both required baselines, and preliminary retrieval results — we are on track. All three exist and are evaluated.

Measured against the proposal's internal schedule we are behind on two items. RRF fusion was scheduled for Milestone 2 (Oct. 13–26) and is not implemented. Preparation of the secondary benchmark was scheduled for Milestone 1 (Oct. 1–12) and has not begun. Both slipped for the same reason: establishing a defensible evaluation methodology took longer than planned. QASPER annotates evidence as free text at paragraph granularity while retrieval operates on overlapping chunks, and resolving that mismatch correctly — rather than reporting a number that silently double-counts overlapping chunks — consumed the time budgeted for fusion. We consider this a worthwhile trade, because every number in the rest of this project depends on that decision being right.

The remaining work, with the deadline held fixed at Dec. 15:

- **RRF fusion** (Nov. 1–5). Fuse the existing BM25 and dense run files. Both are already written in an identical schema over an identical corpus and query set, so fusion consumes run files rather than re-running retrieval.
- **Confidence signals** (Nov. 5–9). Retriever agreement, measured as overlap between the two top-K lists, and ranking confidence, measured as score separation among the top fused candidates.
- **Cross-encoder integration and always-rerank baseline** (Nov. 9–16).
- **Adaptive policy and threshold tuning** (Nov. 16–23). Thresholds are tuned on the train split and then frozen.
- **Secondary benchmark, BEIR-SciFact** (Nov. 16–23), in parallel.
- **Answer generation and QA accuracy** (Nov. 23–30).
- **Held-out evaluation on the QASPER dev split and final report** (Dec. 1–15). The dev split is untouched so far and stays untouched until all settings are frozen.

To absorb the slip we narrow scope in one place: the ablation over reranking depth becomes a single fixed candidate pool size rather than a sweep. The five system configurations promised in the proposal are all retained, because the comparison is the contribution.

2 PRELIMINARY RESULTS

2.1 Experimental Setup

Dataset and corpus. QASPER v0.3, train split: 888 papers, 47,770 paragraphs, 50,189 chunks of 180 words with 30 words of overlap. Both baselines read the identical `corpus.jsonl`, verified by SHA-256 against the recorded build manifest, so the comparison is fair by construction rather than by convention. Retrieval searches all 888 papers; a question's parent paper is treated as provenance and never as a candidate filter.

Evaluation subset. QASPER v0.3 is publicly released as train and dev only, so we use all 2,593 train questions for development and reserve the 1,005 dev questions as a held-out set to be used after all settings are frozen. Of the 2,593 train questions we score the 1,836 (70.8%) whose every evidence entry mapped unambiguously to a prepared paragraph. The 757 excluded questions break down as follows, one mutually exclusive reason each: 271 unanswerable, 7 with annotator disagreement over answerability, 85 with at least one annotation carrying no evidence, 319 whose evidence points at a figure or table that is not part of the text corpus, 69 whose evidence is a section heading, and 6 whose evidence text matches more than one paragraph. Of those 757, 262 do have some mapped evidence but fail the strict completeness condition; excluding them keeps the recall denominator honest, at the cost of a smaller sample. Eligible questions have 1.605 gold paragraphs on average.

Credit granularity. Gold evidence is annotated per paragraph, and our chunks overlap, so two retrieved chunks can belong to one gold paragraph. Scoring them as two hits would inflate recall. We therefore fold each retrieved chunk onto its source paragraph, keep each paragraph's best rank, and then score at paragraph granularity: k counts distinct paragraphs. We also report the same cutoffs applied at chunk depth as a cross-check; the two agree at $k = 100$, as they must.

Metrics. Recall@k for $k \in \{1, 5, 10, 20, 100\}$, averaged over eligible questions; MRR@10 using the rank of the first gold paragraph; hit rate@k, the fraction of questions with at least one gold paragraph in the top k; and per-query latency. Latency percentiles are recomputed by the evaluator with a single nearest-rank estimator over one measurement per question, because each retrieval script records its own percentiles with a different estimator and those are not mutually comparable.

Systems compared. BM25, our original implementation, $k_1 = 1.5$, $b = 0.75$, Unicode word tokenizer, average document length 71.9 tokens. Dense retrieval, `BAAI/bge-m3`, 1024 dimensions, FAISS `IndexFlatIP` over normalized vectors. Both retrieve the top 100 chunks per question with no reranking and no query rewriting.

Hardware. One Apple Silicon laptop, 10 CPU cores, 32 GB unified memory, PyTorch 2.15 on the Metal backend. Embedding all 50,189 chunks took 37.6 minutes. Dense latency is measured after a warm-up query; BM25 has no warm-up, which if anything favors BM25 in the table below.

Table 1: Retrieval quality on the QASPER train split, paragraph-level credit over 1,836 eligible questions. Both systems use the identical corpus, chunking, and query set.

Metric	BM25	Dense (bge-m3)
Recall@1	0.0262	**0.0594**
Recall@5	0.1018	**0.1493**
Recall@10	0.1419	**0.1915**
Recall@20	0.1918	**0.2353**
Recall@100	0.2924	**0.3481**
MRR@10	0.0742	**0.1165**
Hit rate@10	0.1716	**0.2271**
Hit rate@100	0.3426	**0.4020**
Latency p50 (ms)	**9.36**	23.67
Latency p95 (ms)	**17.76**	24.94

2.2 Results

2.3 Discussion

Dense retrieval leads BM25 on every quality metric, and the margin is widest where it matters most. At $k = 1$ dense retrieval is 2.3× better; by $k = 100$ the ratio has fallen to 1.19×. MRR@10 shows the same shape, at 1.57×. The two systems are therefore not finding very different evidence so much as ordering it differently: BM25's lexical signal does locate relevant paragraphs, but it buries them deeper in the ranking. This is the single most useful finding for our proposed method. An adaptive reranker cannot recover evidence that is absent from the candidate pool, so Recall@100 bounds what reranking can achieve — 0.3481 for dense, 0.2924 for BM25 — while the gap between Recall@100 and Recall@10 is the headroom a reranker can actually exploit. For dense retrieval that headroom is 0.157 absolute, which is larger than Recall@10 itself. Reranking is worth doing, and fusing the two candidate pools should raise the ceiling further.

The cost side of the trade is visible too. Dense retrieval is 2.5× slower at the median, entirely because every query must be encoded; BM25 does no neural work at query time. Dense latency is also far more predictable: its p95/p50 ratio is 1.05 against BM25's 1.90, since BM25's cost scales with how many postings lists a query touches. This is exactly the regime our adaptive policy targets, and it tells us the policy should be judged on cross-encoder invocations avoided rather than on retrieval latency, which is already small relative to reranking.

Absolute values are modest, and the reason is the setup rather than the systems. Each question is answered against all 888 papers with no same-paper filter, so a system must first find the right paper among 50,189 chunks. QASPER questions are also written by readers of a specific paper and are frequently generic in isolation — "which baselines did they compare against" — which gives a lexical retriever little to work with across papers. We expect this to be where hybrid fusion helps most, since the two retrievers fail on different questions.

Two implementation challenges are worth recording. First, the proposal's intended dense encoder, `all-MiniLM-L6-v2`, accepts 512 tokens, while our longest chunk is 1,061 tokens; silently truncating a third of a chunk would have made the dense baseline unfair rather than merely weak. We switched encoders and made the index builder refuse to run if any chunk would be truncated, so the failure cannot recur unnoticed. Second, our first evaluator draft credited every chunk independently, which counted overlapping chunks of one gold paragraph as separate evidence and overstated recall. The paragraph-level fold in Section 2.1 fixes this, and a regression test now asserts that two overlapping chunks of one gold paragraph yield a recall of 1/1 rather than 2/1. A second test asserts that BM25's zero-score padding rows, which appear when fewer than 100 chunks share any term with the query, cannot earn recall.

3 EVOLVING METHODOLOGY

3.1 Updated Plan

The architecture is unchanged: retrieve with BM25 and dense retrieval, fuse with Reciprocal Rank Fusion, estimate the confidence of the fused ranking from retriever agreement and score separation, and invoke a cross-encoder reranker only when confidence is low. The contribution remains the adaptive policy that decides when reranking is necessary. What changed is that every component is now a committed choice rather than an example, and the evaluation protocol is specified precisely.

Components reused.

- ~~BM25 implementation e.g. `rank-bm25`~~ → BM25 implemented by us directly, Python standard library only, $k_1 = 1.5$, $b = 0.75$
- ~~pretrained dense model e.g. all-MiniLM-L6-v2~~ → dense encoder `bge-m3`, 1024 dimensions, 8,192-token limit
- ~~similarity search e.g. FAISS~~ → FAISS `IndexFlatIP` over normalized vectors, exact cosine search, no approximation
- ~~cross-encoder e.g. `ms-marco-MiniLM-L6-v2`~~ → cross-encoder `ms-marco-MiniLM-L6-v2`, applied to a fixed candidate pool of the top 50 fused chunks
- ~~LLM for answer generation e.g. OpenAI API using a fixed ChatGPT model~~ → answer generation by `qwen3:8b` from the Qwen3 family, served locally with Ollama, temperature 0, one fixed prompt across all configurations
- compute platform: one Apple Silicon laptop, 32 GB unified memory, PyTorch Metal backend. Embedding the 50,189-chunk corpus takes 37.6 minutes and the cross-encoder is a 22M-parameter model, so GPU embedding and reranking do not require PACE. PACE-ICE remains our fallback if the secondary benchmark proves larger than this machine can handle.

Chunking and retrieval setup. Documents are split into chunks of 180 words with 30 words of overlap, and a chunk never crosses a source paragraph boundary, so each chunk belongs to exactly one paragraph and carries its identifier. Retrieval searches the entire split; a question's parent paper is provenance, not a candidate filter.

Mapping retrieved chunks to gold evidence. QASPER annotates gold evidence at paragraph granularity. For Recall@K and MRR we fold each retrieved chunk onto its source paragraph, keeping

that paragraph's best rank, and score the resulting deduplicated paragraph ranking against the gold paragraph set, so K counts distinct paragraphs. Overlapping chunks of one gold paragraph count once. We additionally report the same cutoffs at chunk depth as a cross-check.

Evaluation set. ~~For QASPER, we plan to evaluate approximately 1,000 questions sampled from the designated test set using a fixed random seed.~~ QASPER v0.3 is publicly released as train and dev only, with no evidence-annotated test split. We therefore develop on all 2,593 train questions and hold out all 1,005 dev questions for final evaluation after the adaptive thresholds are frozen. Within a split we score every question whose evidence mapped completely and unambiguously — 1,836 of 2,593 on train, 646 of 1,005 on dev — and report the eligible count together with a mutually exclusive exclusion reason for every remaining question. No random sampling is involved, so no seed is needed.

Metrics. ~~We will report Recall@1, Recall@5, and Recall@10.~~ We report Recall@1, @5, @10, @20, and @100, plus MRR@10 and hit rate@K. The deeper cutoffs are not decoration: Recall@100 is the ceiling that reranking cannot exceed, and the gap between Recall@100 and Recall@10 is the headroom the adaptive policy is trying to convert into quality, so both are needed to interpret our own method. Latency percentiles are recomputed by the evaluator with one nearest-rank estimator across all systems.

3.2 Justification of Changes

Dense encoder. `all-MiniLM-L6-v2` accepts 512 tokens. Our longest chunk is 1,061 tokens, so the proposal's encoder would have silently discarded text from the longest chunks and made the required dense baseline unfairly weak. `bge-m3` accepts 8,192 tokens, so no chunk is truncated, and the index builder now refuses to run if any chunk would exceed the limit.

BM25 implemented directly. The project requires non-trivial original implementation rather than configuration of existing components, and writing BM25 ourselves also let us make tie-breaking deterministic and bring the index into the same checksum-verified reproducibility chain as everything else. It costs little: index construction over 50,189 chunks takes 1.9 seconds.

Evaluation set. This change was forced rather than chosen. The proposal assumed an evidence-annotated QASPER test split; the public v0.3 release has train and dev only. Holding out dev preserves the intent — tune on one set, report on another that was never used for tuning.

Eligibility policy. 757 of 2,593 train questions cannot support automatic paragraph-level evaluation, most often because the annotated evidence is a figure or table that is not in the text corpus at all, or because the question is marked unanswerable. Scoring those questions would penalize every system for evidence no text retriever could return. We exclude them, state the count, and give one reason per excluded question so the denominator is auditable.

Paragraph-level credit. Implementation experience drove this. Our first evaluator counted chunks independently and overstated recall, because overlapping chunks of a single gold paragraph were credited as multiple pieces of evidence. Since QASPER's annotations are paragraph-level, paragraph-level credit is the measurement the annotations actually support.

Metric depth. Added after seeing the baseline numbers. With dense Recall@10 at 0.1915 and Recall@100 at 0.3481, the headroom available to reranking is larger than the quality already achieved, which is a central fact about our method and would have been invisible at the proposal's cutoffs.

3.3 Response to Proposal Feedback

"Specify the exact chunking strategy and the QASPER retrieval setup . . . state how retrieved chunks will be mapped back to gold paragraphs for Recall@K and MRR." Addressed and implemented. Chunks are 180 words with 30 words of overlap and never cross a paragraph boundary; retrieval searches all 888 papers of the split with no same-paper filter. Retrieved chunks are folded onto their source paragraph, keeping the best rank, and scored against the gold paragraph set, so overlapping chunks of one gold paragraph count once and K counts distinct paragraphs; chunk-depth figures are reported alongside as a cross-check. Section 2.1 also states the eligible question count and every exclusion.

"Commit to concrete choices where the proposal currently says 'e.g.' . . . name the LLM and how you will access it . . . and the compute platform for GPU embedding and reranking." Addressed. BM25 is our own implementation rather than a library; the dense encoder is `BAAI/bge-m3` with FAISS `IndexFlatIP`; the cross-encoder is `cross-encoder/ms-marco-MiniLM-L6-v2` over a fixed pool of 50 fused candidates; answer generation uses `qwen3:8b` from the Qwen3 family, served locally with Ollama at temperature 0 with one fixed prompt for all five configurations, which keeps the generator identical across systems and avoids an external API dependency. On compute: all work so far ran on one Apple Silicon laptop using the PyTorch Metal backend, where embedding the full 50,189-chunk corpus took 37.6 minutes, and the cross-encoder has 22M parameters, so GPU embedding and reranking do not require PACE. We keep PACE-ICE as a documented fallback for the secondary benchmark.

4 POTENTIAL RISKS AND MITIGATION

4.1 Risks Already Addressed

Unfair comparison between systems. The whole project rests on comparing five configurations, so any difference in corpus, chunking, or query set between them would invalidate every conclusion. This is now enforced rather than promised. Each index records the checksum of the corpus it was built from; each run records the checksum of the query file and of the index manifest; the evaluator verifies the entire chain and refuses to score a run produced from a different query set, and it scores all systems in one invocation so they necessarily share one eligible set and one latency estimator. An independent rebuild of the train split reproduced the recorded corpus checksum exactly.

Ambiguity in what counts as a correct retrieval. Resolved by folding chunks onto paragraphs, fixing a single documented eligibility policy, and pinning both with regression tests. Two of those tests

encode mistakes we actually made: crediting overlapping chunks separately, and letting BM25's zero-score padding rows earn recall.

Compute availability for dense retrieval. Initially a schedule risk, since the dense baseline was the only component needing a GPU. Measured rather than assumed: 50,189 chunks embed in 37.6 minutes on a laptop GPU backend, so no shared-cluster queue sits on our critical path.

4.2 Remaining Risks

Risk 1: the adaptive policy may not beat always reranking, or dense retrieval alone, by a useful margin. The baselines leave real headroom — dense Recall@10 is 0.1915 against Recall@100 of 0.3481 — but a cross-encoder has to convert that headroom into ranking quality, and the confidence signals have to identify the queries where reranking pays. Either step can fail. The policy is also only interesting if it skips a meaningful share of queries while giving up little quality; a policy that reranks 95% of queries is not a contribution.

Risk 2: the remaining components may not fit in the time left. Four components remain (fusion, reranking with the adaptive policy, the secondary benchmark, answer generation) against six weeks, and we are already behind on fusion and the secondary benchmark.

4.3 Mitigation Plan

For Risk 1. We report the quality-versus-reranking-rate trade-off as a curve over threshold values rather than as a single operating point, so the result is informative even if no threshold dominates always-reranking. The comparison against always-rerank and against hybrid-without-rerank isolates where any gain comes from. If the adaptive policy does not beat always-reranking on quality, the honest and still-publishable result is the trade-off curve plus an analysis of which confidence signal carried information and which did not; retriever agreement and score separation are measured separately so we can say which failed. If the cross-encoder itself turns out not to help, we fall back to reporting fusion against both baselines and document the negative reranking result with per-query evidence from the existing per-question output files.

For Risk 2. The components are ordered so that each one is reportable on its own: fusion first, since it needs no new model and consumes the run files we already have; then reranking and the adaptive policy, which is the contribution; then the secondary benchmark; then answer generation, which the project specification treats as supplementary. If time runs out, we cut from the back. The secondary benchmark can be reduced to a fixed sampled subset of SciFact with the sampling procedure reported, and answer generation can be reported on a sample of questions rather than all of them, since QA accuracy is a secondary metric and the generator is held constant across systems either way. Our evaluation harness already accepts any number of run files in one invocation, so adding each new configuration to the results table costs one command rather than new analysis code.